System Design & Architecture — Interview Questions & Answers

> A comprehensive guide covering SOLID principles, distributed systems, microservices, databases, and architectural patterns.

Table of Contents

1. SOLID Principles
2. Object-Oriented Design & Design Patterns
3. Distributed Systems Core Concepts
4. Microservices Architecture
5. Database & Storage Systems
6. Caching & Performance
7. Message Queues & Event-Driven Architecture
8. System Design Approaching the Interview

---

1. SOLID Principles

Q1.1: What are the SOLID principles? Explain each one.

Answer: SOLID is an acronym for 5 design principles intended to make software designs more understandable, flexible, and maintainable.

1. **Single Responsibility Principle (SRP):** A class should have one, and only one, reason to change. Meaning that a class should only have one job or responsibility.
2. **Open/Closed Principle (OCP):** Objects or entities should be open for extension but closed for modification. You should be able to add new functionality without changing existing code (e.g., using interfaces or abstract classes).
3. **Liskov Substitution Principle (LSP):** Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program. A subclass must override the base class methods in a way that does not break functionality from a client's point of view.
4. **Interface Segregation Principle (ISP):** Many client-specific interfaces are better than one general-purpose interface. No client should be forced to depend on methods it does not use.

5. **Dependency Inversion Principle (DIP):** Depend upon abstractions, not concretions. High-level modules should not depend on low-level modules; both should depend on abstractions.

Q1.2: Give a practical example of violating and fixing the Open/Closed Principle (OCP).

Answer: Violation: A `NotificationService` class with a `sendNotification` method that checks the type (`if type == 'email' ... else if type == 'sms' ...`). If we add 'push' notifications, we must modify the existing class. **Fix:** Create a `Notification` interface with a `send()` method. Create concrete classes `EmailNotification` , `SMSNotification` , and `PushNotification` implementing the interface. The `NotificationService` takes a `Notification` object and calls `.send()` , allowing us to add new notification types without changing `NotificationService` .

---

2. Object-Oriented Design & Design Patterns

Q2.1: Explain the Singleton pattern and its drawbacks.

Answer: The Singleton pattern ensures that a class has only one instance and provides a global point of access to it.
Drawbacks:

- It introduces global state into an application, making testing difficult (dependencies cannot be easily mocked).
- It violates the Single Responsibility Principle by controlling its own creation and lifecycle.
- In multi-threaded environments, lazy initialization of a Singleton requires careful synchronization (e.g., double-checked locking) to prevent multiple instances from being created.

Q2.2: When would you use a Factory Pattern?

Answer: A Factory Pattern is used when:

- The exact types and dependencies of the objects you want to create are unknown beforehand.
- You want to decouple the object creation logic from the system that uses the object.
- You want to localize the knowledge of which class gets instantiated. (e.g., parsing different types of documents like PDF, Word, XML based on file extension).

---

3. Distributed Systems Core Concepts

Q3.1: What is the CAP Theorem?

Answer: The CAP Theorem states that a distributed data store can simultaneously provide at most two out of the following three guarantees:

- **Consistency:** Every read receives the most recent write or an error. All nodes see the same data at the same time.

- **Availability:** Every request receives a (non-error) response, without the guarantee that it contains the most recent write.
- **Partition Tolerance:** The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes. Since network partitions (P) are a given in distributed systems, architects must trade off between Consistency (CP) and Availability (AP).

Q3.2: Explain Eventual Consistency vs. Strong Consistency.

Answer:

- **Strong Consistency:** After a write completes, any subsequent read will return the updated value. This often requires locking and can impact availability/latency (e.g., traditional RDBMS).
- **Eventual Consistency:** If no new updates are made to a given data item, eventually all accesses to that item will return the last updated value. It prioritizes availability and low latency (e.g., DNS, Amazon DynamoDB in eventual mode).

---

4. Microservices Architecture

Q4.1: Microservices vs. Monolith: When to choose which?

Answer:

- **Monolith:** Best for small teams, simple applications, fast initial iteration, and domains with tightly coupled logic. Easier to deploy, test, and debug initially.
- **Microservices:** Best for large teams (enabling independent scaling and deployments), complex domains that can be naturally bounded, applications requiring diverse technology stacks, and systems needing high availability (failure isolation). Drawbacks include operational complexity, network latency, and distributed data management.

Q4.2: How do microservices communicate with each other?

Answer:

- **Synchronous:** REST over HTTP, gRPC (using Protocol Buffers). Pros: Immediate response, easy to understand. Cons: Tight coupling, cascading failures, blocks thread.
- **Asynchronous:** Message queues (RabbitMQ, SQS) or Event brokers (Kafka). Pros: Decoupling, fault tolerance, handles traffic spikes (buffering). Cons: Eventual consistency, complex debugging/tracing.

Q4.3: What is the Saga Pattern?

Answer: The Saga pattern is a sequence of local transactions. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga. If a local transaction fails, the saga executes a series of **compensating transactions** that undo the changes that were made by the preceding local transactions. It is used to maintain data consistency across microservices without distributed transactions (2PC).

5. Database & Storage Systems

Q5.1: Relational (SQL) vs. Non-Relational (NoSQL) Databases.

Answer:

- **SQL (PostgreSQL, MySQL):** Structured data, predefined schema, ACID transactions, strong consistency. Good for complex queries, financial systems, and structured relationships. Scales vertically (usually).
- **NoSQL (MongoDB, Cassandra, DynamoDB):** Unstructured/semi-structured data, dynamic schema, eventual consistency (BASE), horizontally scalable.
  - *Key-Value:* Redis, DynamoDB. Fast lookups.
  - *Document:* MongoDB. Flexible schemas.
  - *Columnar:* Cassandra. High write throughput, time-series data.
  - *Graph:* Neo4j. Highly connected data (social networks).

Q5.2: What is Database Sharding?

Answer: Sharding is a method of horizontal partitioning that splits a large database into smaller, faster, more easily managed parts called data shards. Each shard is held on a separate database server instance, to spread the load.

- **Challenges:** Joins across shards are slow/impossible, transactions across shards are complex, resharding (rebalancing data) when a shard gets full is difficult.

---

6. Caching & Performance

Q6.1: What are common Cache Eviction Policies?

Answer:

- **LRU (Least Recently Used):** Discards the least recently used items first. Most common general-purpose cache policy.
- **LFU (Least Frequently Used):** Discards the items accessed least often. Good for items with a distinct popularity curve.
- **FIFO (First In First Out):** Discards the oldest items first, regardless of access patterns.

Q6.2: Describe Cache Penetration, Cache Breakdown, and Cache Avalanche.

Answer:

- **Cache Penetration:** A request for a non-existent key goes through the cache directly to the DB. If attackers request many non-existent keys, the DB can crash. *Fix: Cache null values with short TTL, or use a Bloom Filter.*
- **Cache Breakdown:** A highly accessed ("hot") key expires, and simultaneously a massive number of requests hit the cache, miss, and hit the DB. *Fix: Mutex locks so only one thread fetches from DB to populate cache, or never expire*

the key (background updates).

- **Cache Avalanche:** A large number of cache keys expire at the exact same time, causing a spike in DB queries. *Fix: Add a random jitter (e.g., 1-5 minutes) to the TTL of keys.*

---

7. Message Queues & Event-Driven Architecture

Q7.1: RabbitMQ vs. Apache Kafka. Which one to choose?

Answer:

- **RabbitMQ:** A traditional message broker (smart broker, dumb consumer). Messages are pushed to consumers and deleted once acknowledged. Excellent for task queues, complex routing (exchanges), and point-to-point communication.
- **Apache Kafka:** A distributed event streaming platform (dumb broker, smart consumer). Events are appended to a log and persisted for a configured time. Multiple consumer groups can read the same stream independently. Excellent for high-throughput stream processing, event sourcing, and log aggregation.

Q7.2: What is the Outbox Pattern?

Answer: It solves the problem of atomically updating the database and sending messages to a message broker. Instead of directly publishing a message (which might fail after a DB commit), the service writes the event to an `outbox` table in the *same* database transaction as the business entity update. A separate background process (or CDC like Debezium) reads the outbox table and reliably publishes the events to the broker.

---

8. System Design Approaching the Interview

Q8.1: What are the steps to approach a System Design interview question?

Answer:

1. **Requirements Clarification (5 mins):** Define functional (what the system does) and non-functional requirements (scale, latency, availability).
2. **Back-of-the-envelope Estimation (5 mins):** Estimate QPS (Queries Per Second), storage, and bandwidth to understand the scale.
3. **High-Level Design (10 mins):** Draw the core components (Client, Load Balancer, Web Servers, Database) and data flow.
4. **Detailed Design (15 mins):** Deep dive into 2-3 key components. Discuss database schema, caching, partitioning, API design.
5. **Identify Bottlenecks & Resolve (10 mins):** Discuss single points of failure, scalability bottlenecks, and how to fix them (e.g., adding caches, sharding, async processing).

> **Last Updated:** July 2026 | **Focus:** System Design & Architecture